Workshop 2 - Event-Oriented Dish Recommendation System: Technical Architecture and Bronze/Silver Pipeline

Jairo Arturo Barrera Mosquera
Dept. of Computer Engineering
Universidad Distrital Francisco José de Caldas
Email: jabarreram@udistrital.edu.co

Maria del Pilar Pradilla Cely
Dept. of Computer Engineering
Universidad Distrital Francisco José de Caldas
Email: mdpilarp@udistrital.edu.co

David Santiago Aldana Gonzalez
Dept. of Computer Engineering
Universidad Distrital Francisco José de Caldas
Email: dsaldanag@udistrital.edu.co

***Abstract*—Food selection in high-competition gastronomic environments such as New York City poses a critical challenge for event planners who must reconcile a large recipe space with rapidly shifting culinary trends and diverse attendee preferences. This paper presents the technical architecture and first functional implementation of a Bronze/Silver ingestion pipeline for an event-oriented dish recommendation system, integrating structured recipe data from the Spoonacular API with unstructured culinary news from the Eater NY RSS feed, orchestrated by Apache Airflow inside a containerised Docker Compose environment following the Medallion layered architecture. Initial results demonstrate successful end-to-end ingestion producing datetime-stamped raw JSON files in the Bronze layer and validated, schema-enforced Parquet files in the Silver layer, establishing a traceable and idempotent foundation for downstream NLP-driven trend analysis and dish recommendation.**

***Index Terms*—data pipeline, Apache Airflow, Medallion architecture, Parquet, recipe recommendation, NLP, Docker, sentiment analysis, event planning**

I. Executive Summary

This report documents the technical architecture and workflow implementation of a data-driven dish recommendation system aimed at event organizers operating in high-density food markets, such as New York City. The technical approach centers on the fusion of structured recipe metadata from the Spoonacular Food API with unstructured editorial content from the Eater NY RSS feed, applying Natural Language Processing (NLP) techniques to extract sentiment signals and identify culinary trends relevant to event-oriented menu curation.

The pipeline design follows a Medallion architecture comprising three distinct layers: a **Bronze** layer that persists raw JSON files from both ingestion sources using datetime-based nomenclature; a **Silver** layer that applies a documented preprocessing pipeline—including null handling, text normalisation, deduplication, and schema enforcement—to produce timestamped Parquet files; and a **Gold** layer (planned for Workshop 3) where aggregated, analysis-ready datasets will be produced using PySpark. Orchestration is handled by two Apache Airflow DAGs implemented with the TaskFlow API. The Bronze ingestion DAG extracts data on a scheduled basis, while the Silver processing DAG is triggered automatically by a `FileSensor` upon detection of new Bronze files, ensuring an event-driven flow.

Key stack decisions include the use of **Docker Compose** for a containerised local environment, ensuring portability and persistence via volume mounts. **Apache Airflow** was selected for its robust dependency management and lineage-logging capabilities, preserving full traceability between layers. **Pandas** was chosen for row-level transformations and text preprocessing due to its efficiency with moderate batch sizes, while **Parquet** was adopted as the Silver persistence format for its columnar compression and compatibility with downstream analytical processing. The pipeline has been verified end-to-end, confirming that both DAGs execute successfully and produce validated, traceable data outputs.

II. Technical Architecture

The system architecture follows a modular, event-driven design based on the Medallion architecture pattern. This approach ensures data integrity, traceability, and scalability for the recommendation engine. Figure 1 illustrates the end-to-end flow, from external ingestion to processed analytical storage.

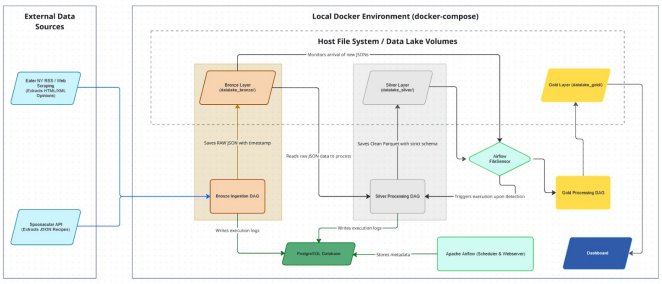

Fig. 1. Full system architecture diagram showing the Medallion layers, data flows, and containerized components.

A. Detailed Component Description

To ensure a robust pipeline, each component has been designed with specific responsibilities and technical constraints:

1) **Spoonacular API (Data Source):**

- *Purpose:* Provides structured gastronomic data (recipes, ingredients, and costs).
- *Technology:* RESTful API, `requests` library.
- *Inputs:* API Key and search parameters (e.g., event-type keywords).
- *Outputs:* Raw JSON objects.
- *Interactions:* Polled by the Bronze DAG via HTTP.

2) **Eater NY RSS (Data Source):**
   - *Purpose:* Captures unstructured culinary trends and news in New York City.
   - *Technology:* XML/RSS, `BeautifulSoup4` for web scraping.
   - *Inputs:* RSS Feed URL.
   - *Outputs:* Raw semi-structured JSON files.
   - *Interactions:* Scraped by the Bronze DAG to extract editorial content.
3) **Apache Airflow (Orchestrator):**
   - *Purpose:* Centralized management of task scheduling and dependency monitoring.
   - *Technology:* Python, TaskFlow API, PostgreSQL (Metadata).
   - *Inputs:* DAG definitions and environment variables.
   - *Outputs:* Execution logs and task status.
   - *Interactions:* Triggers the ingestion tasks and monitors the Bronze directory via `FileSensor` to initiate Silver processing.
4) **Bronze Layer (Raw Storage):**
   - *Purpose:* Immutable landing zone for raw data.
   - *Technology:* Local filesystem mounted via Docker volumes, JSON format.
   - *Inputs:* Raw data from API and RSS tasks.
   - *Outputs:* Datetime-stamped JSON files (`.json`).
   - *Interactions:* Receives data from the Bronze DAG; acts as the trigger source for the Silver DAG.
5) **Silver Layer (Processed Storage):**
   - *Purpose:* Stores cleaned, deduplicated, and schema-enforced data ready for analysis.
   - *Technology:* Pandas, PyArrow, Parquet format.
   - *Inputs:* Raw Bronze JSON files.
   - *Outputs:* Query-optimized Parquet files (`.parquet`).
   - *Interactions:* Consumes Bronze data; provides the foundation for the upcoming Gold layer.

B. Technology Justification

The selection of the technical stack is grounded in the principles of modern data engineering and the specific requirements of the Medallion architecture established in the course:

- **Pandas for Silver Layer Processing:** For the transformation from Bronze to Silver, **Pandas** was selected over alternatives like Polars or Spark due to its superior maturity in handling the complex, nested structures of the **Spoonacular API** and its seamless integration with `BeautifulSoup4` and `NLTK` for the extraction of sentiment from **Eater NY**. Given the moderate batch sizes of our current ingestion cadence, Pandas provides the most efficient balance between development speed and row-level transformation power, particularly for the text-heavy preprocessing required by our recommendation engine.
- **Parquet for Silver Persistence:** Following data engineering best practices, **Parquet** was chosen as the persistence format for the Silver layer. Unlike CSV or JSON, Parquet is a **columnar storage** format that provides efficient compression and, most importantly, **schema enforcement**. This choice ensures that data types are preserved across the pipeline, preventing schema drift and significantly reducing I/O latency when the future recommendation dashboard filters by gastronomic categories or price ranges. This decision directly supports the analytical compatibility required for the **PySpark** joins planned for the Gold layer in Workshop 3.
- **Apache Airflow for Orchestration and Traceability:** Airflow is the core of the pipeline's operational reliability. It was selected to implement the "Event-Driven" design pattern via the **FileSensor** operator. This allows the system to achieve near real-time processing of culinary trends as soon as they land in the Bronze layer. Furthermore, the use of Airflow's **TaskFlow API** enables strict **idempotency** and **lineage-logging**, ensuring that every processed menu in the Silver layer can be traced back to its raw JSON source for auditing and debugging purposes.
- **Docker Compose for Infrastructure Parity:** The use of Docker Compose ensures that the entire environment—including the Airflow scheduler, webserver, and Postgres metadata database—is reproducible across different development machines. By using **Docker Volumes**, we implement a persistence strategy that separates the data lifecycle from the container lifecycle, ensuring that the collected gastronomic data lake persists independently of system restarts, a fundamental requirement for building a historical recommendation database.

C. File Nomenclature Conventions

Strict naming conventions are enforced across both storage layers to ensure full data lineage and idempotency. This strategy allows the system to trace every recommendation in the Silver layer back to its specific source record in the Bronze layer:

- **Bronze Layer (Raw):** `dishName_YYYYMMDD_HHMMSS.json`
  The filename is prefixed with the specific dish or culinary topic, followed by a precise ISO-8601 timestamp. This allows for immediate identification without opening the file.
  Examples: thai_curry_chicken_20260417_143000.json, brooklyn_pizza_trends_20260417_143000.json.
- **Silver Layer (Processed):** `topic_YYYYMMDD_HHMMSS.parquet`
  The prefix represents the functional dataset category (e.g., *menus*, *trends*, or *sentiment*), followed by the timestamp of the processing run.
  Example: foodblog_nycfoodtrends_20260417_143500.parquet.

Rationale for Traceability: By using the `dishName` as the primary identifier in the Bronze layer, the pipeline facilitates granular auditing. If a recommendation in the Gold layer (Workshop 3) shows an anomaly, developers can quickly locate the raw JSON file for that exact dish. The synchronized timestamps prevent file collisions and preserve the full history of the data lake.

III. Preprocessing Pipeline Definition

This section defines the complete data cleaning and transformation pipeline applied to move data from the Bronze layer (raw JSON) to the Silver layer (processed Parquet). All decisions are grounded in data samples collected during Workshop 1 from both the Spoonacular API and the Eater NY RSS Feed. The pipeline is implemented using Pandas and organized into the following stages.

A. Source Schemas and Observed Data Issues

The Spoonacular API returns JSON responses with a top-level `results` array, where each record contains the following fields relevant to the menu recommendation task: `id`, `title`, `summary`, `cuisines`, `dishTypes`, `diets`, `occasions`, `readyInMinutes`, `servings`, `pricePerServing`, `healthScore`, `spoonacularScore`, `vegetarian`, `vegan`, `glutenFree`, `dairyFree`, and `sourceUrl`. From the samples collected in Workshop 1, the following issues were observed: the `cuisines` and `occasions` fields are frequently returned as empty lists, as seen in the sample record where `cuisines` is `[]`; the `preparationMinutes` and `cookingMinutes` fields are returned as `null` in the majority of records; and the `summary` field contains embedded HTML tags and anchor elements that must be stripped before any NLP processing. Fields such as `image`, `imageType`, `gaps`, `license`, `creditsText`, `weightWatcherSmartPoints`, and `spoonacularSourceUrl` are not relevant to the recommendation pipeline and will be dropped during Silver processing.

The Eater NY RSS Feed provides records with the following fields: `article_title`, `article_url`, `article_id`, `author`, `article_summary`, `categories`, `location_focus`, `published_date`, and `date_extracted`. From the Workshop 1 samples, the following issues were observed: the `article_title` field may contain HTML-encoded entities such as `’` that require decoding; the `article_summary` field contains mixed content including image captions, pipe-separated metadata fragments, and promotional text that must be cleaned before NLP processing; and the `categories` field is returned as a list that may contain a single generic label such as `"Press Room"`, offering limited discriminative value for recommendation filtering without normalization.

B. Null Handling

Null handling strategies are defined per field based on the role each field plays in the recommendation pipeline and the issues observed in Workshop 1 samples. For Spoonacular records, the `preparationMinutes` and `cookingMinutes` fields are consistently returned as `null` in the raw Bronze JSON. Rather than dropping these records, null values in both fields are coerced to numeric using `pandas.to_numeric(errors="coerce")` and filled with `-1` as a sentinel value, indicating that the information is unavailable without invalidating the record. All other fields in the Spoonacular records were observed to be present and non-null in Workshop 1 samples and are retained as-is after list flattening and HTML cleaning.

For Eater NY records, all fields observed in Workshop 1 samples — `article_title`, `article_url`, `article_id`, `author`, `article_summary`, `categories`, `location_focus`, `published_date`, and `date_extracted` — were present and non-null. No sentinel filling or record dropping was required at this stage. As the pipeline matures in subsequent workshops, additional null handling rules will be introduced based on a broader sample of ingested records.

C. Data Type Normalization

For Spoonacular records, the `preparationMinutes` and `cookingMinutes` fields are explicitly cast to `int32` after null filling, as they represent discrete time values in minutes. All remaining fields are retained in their native types as parsed from the Bronze JSON by Pandas. Fields returned as lists by the API — specifically `cuisines`, `dishTypes`, `diets`, and `occasions` — are flattened into comma-separated strings using a `flatten_list_columns()` pass that detects list-valued columns automatically and joins their elements. This conversion ensures full compatibility with the Parquet format, which does not natively support mixed-type list columns in all writer configurations.

For Eater NY records, all fields are retained in their native string and list types as parsed from the Bronze JSON. The `categories` field, returned as a list, is likewise flattened to a comma-separated string by the same `flatten_list_columns()` function applied uniformly across both sources.

D. Text Preprocessing for NLP

Text preprocessing targets the fields that carry the descriptive content used by the recommendation engine. For Spoonacular records, the `summary` field is processed by stripping all embedded HTML tags and anchor elements using `BeautifulSoup4`'s `get_text()` method with whitespace separator, producing a clean plain-text summary. This step is applied via the `strip_html_tags()` function, which falls back to a regular expression substitution (`re.sub(r"<[^]+>", " ", text)`) if

BeautifulSoup4 is not available in the execution environment.

For Eater NY records, the article_summary field undergoes a full NLP cleaning pipeline implemented in the clean_nlp_text() function. The steps applied sequentially are: first, HTML entity decoding using html.unescape() to resolve encoded characters such as ’ observed in Workshop 1 titles and summaries; second, lowercasing of all text; third, punctuation removal via regular expression substitution retaining only alphanumeric characters and whitespace; fourth, normalization of consecutive whitespace to single spaces; and fifth, stop word removal using the English stop word list from NLTK, downloaded automatically if not present in the environment. The result is stored in a new field article_summary_clean, preserving the original article_summary field for traceability to the Bronze source.

E. Outlier Detection and Handling

In this first version of the Silver pipeline, explicit outlier detection routines are not yet applied. The Workshop 1 sample size for both sources was insufficient to establish robust distributional baselines for fields such as readyInMinutes, servings, and pricePerServing. As established in the workshop guidelines, the preprocessing pipeline implemented here is a first version, and outlier detection strategies based on IQR for right-skewed fields such as readyInMinutes and pricePerServing, and Z-score for more symmetric fields such as servings, will be incorporated in Workshop 3 once a sufficient volume of ingested records is available to compute stable statistics.

F. Deduplication

Duplicate records may arise across ingestion batches when the same Spoonacular recipe or Eater NY article is retrieved in multiple pipeline runs. For Spoonacular records, deduplication is performed on the id field, which is a unique integer identifier assigned by the API, ensuring that no two records representing the same recipe coexist in the Silver output. For Eater NY records, deduplication is performed on the combination of article_url and published_date, as RSS entries carry a stable URL per article that uniquely identifies each piece of content. In both cases, the most recently ingested version of each duplicate is retained, prioritizing the latest available data from the Bronze source files. This strategy ensures that the Silver layer remains consistent and free of redundant records across successive ingestion runs.

G. Schema Enforcement

The Silver Parquet files produced by this pipeline version contain all fields present in the Bronze JSON records after the transformations described above. For Spoonacular, the output schema retains all original API fields with the following modifications: preparationMinutes and cookingMinutes are cast to int32 with -1 as the null sentinel; cuisines, dishTypes, diets, and occasions are stored as comma-separated strings; and summary is stored as a cleaned plain-text string. For Eater NY, the output schema retains all original fields with the addition of article_summary_clean, which stores the NLP-preprocessed version of article_summary. Both output schemas are enforced implicitly by the Pandas to_parquet() call using PyArrow as the backend, which infers and persists column types from the DataFrame at write time.

IV. Docker Stack Description

The full project stack for the New York menu recommendation pipeline is containerized using **Docker Compose**, providing a reproducible and portable local environment that isolates all pipeline dependencies. The stack is composed of two services: a postgres container serving as the Airflow metadata database, and a single airflow container that consolidates the webserver and scheduler processes.

A. Services

The **postgres** service runs on the official postgres:13 image and provides the relational metadata database required by Airflow to store DAG run history, task states, variables, and connections. It is configured with a dedicated database named airflow and exposed on port 5432. Database contents are persisted across container restarts via a named volume pgdata mounted at /var/lib/postgresql/data.

The **airflow** service is built from a custom Dockerfile located at the project root, which extends the base Airflow image with all Python dependencies required by the pipeline. This service consolidates three responsibilities: database initialization via airflow db upgrade, user creation, and the concurrent execution of the Airflow webserver and scheduler processes via a single bash entrypoint. The webserver is accessible at localhost:8080 and provides the Airflow UI for monitoring DAG runs, task states, and logs. The scheduler continuously parses DAG definitions and triggers task execution according to the schedules defined for the Spoonacular ingestion DAG and the Eater NY scraping DAG. The LocalExecutor is used as the execution backend, which runs tasks as subprocesses within the same container, appropriate for the local development scope of this workshop. The service depends on postgres being available before startup, enforced via the depends_on directive.

B. Environment Configuration

The Airflow service is configured through a combination of environment variables defined directly in the docker-compose.yml and a .env file loaded via the env_file directive. The database connection is set through AIRFLOW__DATABASE__SQL_ALCHEMY_CONN, pointing to the postgres service using the postgresql+psycopg2 driver. Example loading is disabled via AIRFLOW__CORE__LOAD_EXAMPLES: "False" to keep the Airflow UI clean. The Spoonacular API key is injected at runtime through the

SPOONACULAR_API_KEY environment variable, read from the .env file and made available to all DAG tasks without hardcoding credentials in the codebase. The PYTHONPATH is set to /opt/airflow/dags to allow DAG files to import shared Python modules located in the same directory.

C. Volume Mounts

The following local directories are mounted as Docker volumes to ensure that ingested menu data, processed outputs, DAG definitions, and execution logs persist on the host filesystem across container restarts:

- ./airflow/dags → /opt/airflow/dags: Contains all DAG definitions and supporting Python modules. Mounting this directory enables live DAG reloading without requiring container rebuilds.
- ./logs → /opt/airflow/logs: Persists Airflow task execution logs on the host for inspection outside the container.
- ./datalake_bronze → /opt/airflow/datalake/bronze: Persists raw JSON files produced by the ingestion DAGs on the host filesystem.
- ./datalake_silver → /opt/airflow/datalake/silver: Persists processed Parquet files produced by the Silver DAG on the host filesystem.
- ./datalake_gold → /opt/airflow/datalake/gold: Reserved for the Gold layer aggregation outputs planned for Workshop 3.

D. Setup Instructions

To launch the full pipeline stack, the following steps must be executed from the project root directory. First, create a .env file at the project root containing the Spoonacular API key as SPOONACULAR_API_KEY=your_key_here. Second, build the custom Airflow image and start all services by running:

```
docker compose up --build
```

On first startup, the entrypoint automatically initializes the Airflow metadata database, creates a default admin user with username admin and password admin, and launches the webserver and scheduler processes. The Airflow UI becomes accessible at http://localhost:8080 once the webserver is ready. To stop all services and preserve data, run:

```
docker compose down
```

To stop all services and remove all persisted database data, run:

```
docker compose down -v
```

The pipeline data stored in the mounted local directories (datalake_bronze/, datalake_silver/, datalake_gold/) is preserved on the host filesystem regardless of whether -v is used, as these are bind mounts rather than named volumes.

V. DAG Documentation

The orchestration layer of the menu recommendation pipeline is composed of two Apache Airflow DAGs implemented using the TaskFlow API. The first handles Bronze ingestion from both data sources, and the second handles Silver processing triggered by the arrival of new Bronze files.

A. Bronze Ingestion DAG

The Bronze ingestion DAG is defined in bronze_ingestion_dag.py with the identifier bronze_ingestion_pipeline. Its purpose is to extract raw menu and restaurant data from both the Spoonacular API and the Eater NY RSS Feed, and persist the results as JSON files in the Bronze layer of the datalake.

The DAG is scheduled with @daily frequency, starting from January 1, 2024, with catchup=False to prevent backfilling of historical runs. It is tagged with bronze for identification in the Airflow UI.

The DAG contains two tasks defined with the @task decorator. The first task, extract_api_data, invokes the main() function of the scripts.bronze.api_ingestion module, which performs authenticated HTTP requests to the Spoonacular API and persists the response as a JSON file in datalake_bronze/api/. The second task, extract_web_data, invokes the main() function of the scripts.bronze.WebScrapping_NY module, which scrapes the Eater NY RSS Feed using BeautifulSoup4 and persists the extracted records as a JSON file in datalake_bronze/webscraping/. The tasks are chained sequentially, with API extraction completing before web scraping begins, as defined by the dependency extract_api_data() >> extract_web_data().

The expected output of each DAG run is two Bronze JSON files following the naming convention nameOfTheDish_YYYYMMDD_HHMMSS.json, one per source, persisted in their respective subdirectories under datalake_bronze/.

Figure 2 shows the Airflow dashboard with both DAGs, and Figure 3 shows the list of successful DAG runs produced during the workshop validation period.

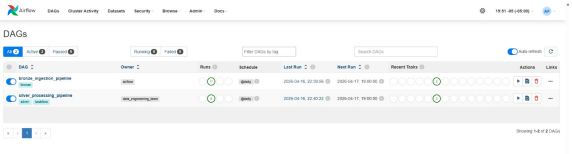

Fig. 2. Airflow UI dashboard showing the bronze_ingestion_pipeline and silver_processing_pipeline DAGs.

B. Silver Processing DAG

The Silver processing DAG is defined in silver_processing_dag.py with the identifier silver_processing_pipeline. Its purpose is to detect the arrival of new JSON files in the Bronze layer and

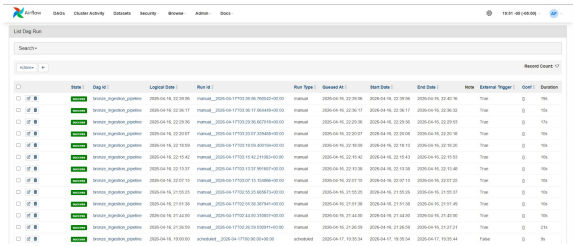

Fig. 3. Airflow List DAG Runs view showing successful execution history of the bronze_ingestion_pipeline.

trigger the full preprocessing pipeline that produces cleaned Parquet files in the Silver layer.

The DAG shares the same `@daily` schedule and `start_date` as the Bronze DAG, with `catchup=False`. It is tagged with `silver` and `taskflow`. Default arguments configure a single retry with a two-minute delay on task failure, assigned to the `data_engineering_team` owner.

The DAG contains two components. The first is a `FileSensor` operator named `wait_for_bronze_files`, configured with `filepath='api/*.json'` and `fs_conn_id='fs_default'`, where the `fs_default` Airflow connection points to `/opt/airflow/datalake/bronze`. The sensor polls the Bronze API directory every 30 seconds (`poke_interval=30`) and times out after 10 minutes (`timeout=600`) if no file is detected. This mechanism ensures that Silver processing is only triggered once new Bronze data is confirmed to be present, decoupling the two DAGs without requiring explicit cross-DAG dependencies.

The second component is the `process_data_task`, a `@task`-decorated function that imports and executes the `main()` function from the `scripts.silver.preprocess_datasets` module. This function applies the full preprocessing pipeline described in Section 2.2, covering list flattening, HTML stripping, NLP text cleaning, type normalization, and Parquet persistence for both Spoonacular and Eater NY records. The task is chained after the sensor via `wait_for_data >> process_data_task()`.

The expected output of each DAG run is one or more Silver Parquet files persisted in `datalake/silver/api/` and `datalake/silver/webscraping/`, following the naming convention `source_YYYYMMDD_HHMMSS.parquet` derived from the originating Bronze filename.

Figure 4 and Figure 5 show the List Task Instance view and the DAG code as viewed in the Airflow UI respectively.

VI. Results Evidence

This section presents the evidence of successful pipeline execution across both the Bronze ingestion and Silver processing stages, validated through the Airflow UI and the datalake directory structure.

[t]0.48

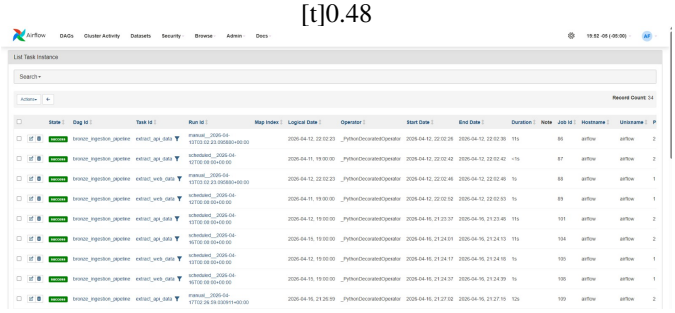

Fig. 4. List Task Instance view showing successful execution of wait_for_bronze_files and process_data_task.

[t]0.48

```python
from airflow.decorators import dag, task
from airflow.sensors.filesystem import FileSensor
from datetime import datetime, timedelta
import os


default_args = {
    'owner': 'data_engineering_team',
    'retries': 1,
    'retry_delay': timedelta(minutes=2),
}

@dag(
    dag_id='silver_processing_pipeline',
    default_args=default_args,
    schedule_interval='@daily',
    start_date=datetime(2024, 1, 1),
    catchup=False,
    tags=['silver', 'taskflow']
)
def silver_processing_dag():


    wait_for_data = FileSensor(
        task_id='wait_for_bronze_files',
        filepath='api/*.json',
        fs_conn_id='fs_default',
        poke_interval=30,
        timeout=600
    )

    @task()
    def process_data_task():

        from scripts.silver.preprocess_datasets import main as run_silver_logic


        run_silver_logic()
        return "Silver layer completed: JSONs converted to Parquet"


    wait_for_data >> process_data_task()


dag_instance = silver_processing_dag()
```

Fig. 5. Silver DAG code in the Airflow UI showing the FileSensor configuration and TaskFlow task definition.

Fig. 6. Silver processing pipeline execution evidence and DAG definition.

A. Airflow UI Evidence

The successful execution of both DAGs was verified through the Airflow UI. Figure 2 shows the main dashboard with both `bronze_ingestion_pipeline` and `silver_processing_pipeline` active and in a healthy state. Figure 3 confirms multiple successful daily runs of the Bronze ingestion pipeline across the workshop validation period. Figure 4 shows the List Task Instance view for the Silver DAG, confirming that both the `wait_for_bronze_files` FileSensor and the `process_data_task` completed successfully in each run.

B. Bronze Layer Output

Each successful run of the `bronze_ingestion_pipeline` produces two JSON files in the Bronze layer: one under `datalake/bronze/api/` containing the raw Spoonacular API response, and one under `datalake/bronze/webscraping/` containing the Eater NY RSS records extracted during that run. Both files follow the naming convention `source_YYYYMMDD_HHMMSS.json`, where the timestamp corresponds to the moment of ingestion. The raw JSON structure is preserved without any transformation, maintaining full fidelity to the original source responses as required by the Bronze layer design.

C. Silver Layer Output

Figure **??** shows the Silver layer directory after several pipeline runs, confirming the successful creation of Parquet files in both $\texttt{datalake}_{s}ilver/api/and$

VII. Challenges and Decisions

The implementation of the Bronze and Silver pipeline presented two main technical challenges, both resolved before the final pipeline validation.

A. FileSensor Connection Configuration

The `FileSensor` operator in the Silver DAG requires an Airflow filesystem connection (`fs_conn_id`) that maps a logical connection identifier to a physical path inside the container. The initial pipeline runs failed at the `wait_for_bronze_files` task because the `fs_default` connection had not been configured in the Airflow metadata database, causing the sensor to be unable to resolve the Bronze directory path. The issue was resolved by manually creating the `fs_default` connection through the Airflow UI under Admin → Connections, setting the connection type to `File (path)` and the path to `/opt/airflow/datalake/bronze`. This configuration is not captured in the `docker-compose.yml` and must be recreated manually after each environment reset, which represents a known limitation of the current setup that will be addressed in Workshop 3 by initializing connections programmatically via an Airflow environment variable or a startup script.

B. Module Import Resolution Inside DAG Tasks

The Bronze and Silver DAGs import supporting Python modules located in `airflow/dags/scripts/`, using paths such as `scripts.bronze.api_ingestion` and `scripts.silver.preprocess_datasets`. During initial testing, these imports failed at task runtime with `ModuleNotFoundError`, even though the modules were present in the mounted DAG volume. The root cause was that the Airflow task execution environment did not include the `/opt/airflow/dags` directory in its `PYTHONPATH` by default. The issue was resolved by explicitly setting `PYTHONPATH: /opt/airflow/dags` in the `environment` section of the `airflow` service in `docker-compose.yml`, and additionally adding `sys.path.append('/opt/airflow/dags')` at the top of the Bronze DAG file as a safeguard. Both measures together ensured consistent module resolution across all task execution contexts.

References

[1] Y. Zhang, M. Chen, Q. Liu, and E. Chen, "Towards explainable conversational recommendation," in *Proc. 28th Int. Joint Conf. Artif. Intell. (IJCAI)*, Macao, China, 2019, pp. 4947–4953.

[2] T. Lippincott and A. Sporleder, "A soft-layer extension for the ACE relation ontology," in *Proc. 10th Int. Conf. Comput. Semantics (IWCS)*, Potsdam, Germany, 2013, pp. 148–159.

[3] A. Sadlon, L. Breschi, and M. Tumminello, "Sentiment analysis of restaurant reviews: a multi-platform comparative study," *Information*, vol. 12, no. 8, p. 308, 2021.

[4] Spoonacular, "Spoonacular Food & Recipe API Documentation," 2024. [Online]. Available: https://spoonacular.com/food-api/docs

[5] Eater New York, "Eater NY RSS Feed," 2026. [Online]. Available: https://ny.eater.com/rss/index.xml

[6] Apache Software Foundation, "Apache Airflow Documentation," v2.9, 2024. [Online]. Available: https://airflow.apache.org/docs/

[7] Apache Software Foundation, "TaskFlow API Tutorial," 2024. [Online]. Available: https://airflow.apache.org/docs/apache-airflow/stable/tutorial/taskflow.html

[8] Databricks, "What is the Medallion Lakehouse Architecture?" 2023. [Online]. Available: https://www.databricks.com/glossary/medallion-architecture

[9] W. McKinney, *Python for Data Analysis*, 3rd ed. Sebastopol, CA: O'Reilly Media, 2022.

[10] Apache Software Foundation, "Apache Parquet Documentation," 2024. [Online]. Available: https://parquet.apache.org/docs/

[11] Docker Inc., "Docker Compose Documentation," 2024. [Online]. Available: https://docs.docker.com/compose/

[12] K. D. Perpetua, "feedparser: Universal Feed Parser," 2024. [Online]. Available: https://feedparser.readthedocs.io/